Universidad Carlos III

Sistemas Distribuidos

2025-26

Práctica Final

GRUPO: **81**

Alumnos: **Lidia Valera López; NIA: 100522236 /Ana Díaz Jiménez; NIA: 100522155**

Correos: **100522236@alumnos.uc3m.es** / **100522155@alumnos.uc3m.es**

ÍNDICE

1. Introducción 2
2. Descripción del código 3
    2.1. Parte 1 3
        - Servidor 3
        - Claves.h 3
        - Concurrencia servidor 3
        - Claves.c 4
        - Cliente 6
        - Protocolo 6
    2.1. Parte 2 7
        - Transferencia de ficheros 7
        - Servicio web 8
        - Protocolo SOAP y Estructura 9
        - Infraestructura de Transporte y Despliegue 9
        - Servicio RPC: Registro de actividad (Logs) 10
3. Descripción compilación y obtener ejecutables 10
4. Batería de pruebas 11
5. Conclusiones 14

1. Introducción

En esta práctica se desarrolla un servicio de envío de mensajes entre usuarios, similar a WhatsApp pero más sencillo. Este servicio es distribuido y permite el registro de usuarios, el envío y recepción de mensajes, la transferencia directa de archivos entre clientes (la comunicación entre múltiples usuarios) mediante una arquitectura cliente-servidor basada en sockets, RPC y Servicios Web, utilizando además distintos lenguajes de programación (C y Python). La aplicación está compuesta por las siguientes partes:

- Servicio de mensajería: es el servidor del programa y se encarga de gestionar el registro de usuarios, las conexiones y sirve de intermediario para la entrega de mensajes entre los usuarios.
- Usuarios: son los clientes del programa e interactúan con el sistema para enviar mensajes y transferir ficheros entre ellos.
- Servicio de registro: está basado en RPC y se encarga de monitorizar las operaciones que van realizando los diferentes usuarios.
- Conversor de mensajes: lo implementamos mediante servicios web para la normalización de textos que se envían los usuarios. De esta forma, los mensajes que se envíen pasarán por este conversor para eliminar los espacios en blanco repetidos.

Toda esta aplicación la implementamos en 2 partes. En la primera parte creamos un servidor concurrente multihilo (en C), que gestiona el registro, conexión, desconexión y baja de usuarios. Además este servidor se encargará de almacenar y reenviar mensajes cuando el destinatario no esté disponible. Por otro lado, los clientes también son multihilo (implementados en Python) para poder enviar peticiones al servidor mientras escuchan la llegada de mensajes de otros usuarios. La comunicación entre el cliente y el servidor se rige por un protocolo de comunicación basado en sockets TCP y cadenas terminadas en '\0'.

En la segunda parte, incorporamos el envío de mensajes con archivo adjunto (SENDATTACH). Además este archivo se envía directamente de un cliente a otro, sin que intervenga el servidor, utilizando el comando GETFILE. Para que esto sea posible, el servidor le proporciona al usuario que quiere mandar el archivo, la dirección IP y puerto del usuario al que se lo quiere enviar. Además integramos un servicio web en Python que se encarga de la conversión de mensajes explicado anteriormente. Por último implementamos un servicio RPC que el servidor invoca cada vez que un usuario realiza una operación, registrando el nombre del usuario y la operación en un log.

2. Descripción del código

2.1. Parte 1

- **Servidor**

El servidor se encuentra en el fichero servidor.c, donde gestiona las conexiones, la persistencia en memoria de los usuarios y la coordinación de los hilos, los cuales no los generamos bajo demanda sino que tenemos un pool de 8 hilos. Además el servidor tiene la lógica de las acciones que puede realizar el usuario en claves.c, junto con su cabecera claves.h.

- **Claves.h**

Dentro de claves.h tenemos dos estructuras de datos, estas son “User” y “Message”.

User representa a un usuario registrado y contiene los campos:
- “name”: nombre de usuario que lo identifica.
- “ip” y “port”: dirección IP y puerto de escucha del cliente si el cliente está conectado.
- “status” : vale 0 si el cliente está desconectado o 1 si está conectado.
- “last_msg_id”: es el último identificador del mensaje enviado por el usuario que usamos para garantizar que los IDs sean secuenciales por usuario.
- “pending_msgs”: es una lista enlazada de mensajes pendientes de entregar al usuario.
- “next”: es un puntero que apunta al siguiente usuario en la lista global user_list.

Message representa un mensaje pendiente y contiene los campos:
- “id”: identificador del mensaje, asignado por el servidor.
- “sender”: es el nombre del remitente.
- “content”: es el cuerpo del mensaje que puede ser como máximo 255 caracteres.
- “filename”: es el nombre del archivo adjunto, que en esta primera parte dejamos vacío.
- “next”: es un puntero que apunta al siguiente mensaje pendiente de la lista.

- **Concurrencia servidor**

El servidor está diseñado para gestionar múltiples peticiones de forma simultánea garantizando la seguridad de los hilos y la integridad de los datos.
El servidor cuenta con una lista global de usuarios (user_list), la cual está protegida con un mutex global ya que todos los hilos acceden a esta lista y así evitamos condiciones de carrera como por ejemplo haya dos hilos que modifiquen el estado de un usuario al mismo tiempo.
El servidor también tiene una cola de tareas circular compartida, la cual también está protegida con un mutex específico para esta cola para garantizar la exclusión mutua y evitar condiciones de carrera, ya que todos los hilos acceden a esta estructura. De esta

forma aseguramos que las operaciones de inserción y extracción de sockets sean atómicas.
En cuanto a los hilos, en vez de crear uno por cada petición, empleamos un pool fijo de 8 hilos. Esta arquitectura sigue el modelo Productor-Consumidor, en el que el productor es el hilo principal y los consumidores son los hilos trabajadores (los 8 hilos del pool). El hilo principal ejecuta un bucle infinito en el que aceptamos una petición y bloqueamos la ejecución hasta que llega una nueva conexión. Al recibirla, utilizamos la estructura task_t que almacena el socket del cliente y su dirección IP, y llamamos a la función enqueue(). Esta función, junto con dequeue(), las utilizamos para añadir y extraer tareas de la cola. Estas funciones están sincronizadas con un mutex y una variable de condición para que los hilos esperen cuando no hay trabajo (la cola esté vacía) y no haya condiciones de carrera. Así cada vez que el hilo principal inserta una tarea, se envía una señal mediante la variable de condición para despertar a uno de los hilos del pool y asignarle una nueva tarea. Los hilos trabajadores, ejecutan la función thread_function(), en la que se extrae una tarea y se procesan todas las operaciones que ese cliente envíe sobre la misma conexión.

- **Claves.c**

Todas las acciones que pueden realizar los usuarios se encuentran implementadas en este archivo. Estas funciones leen los parámetros del socket utilizando la función auxiliar "read_str()", que lee carácter a carácter hasta un fin de linea (\0). Las funciones son las siguientes:

Nombre función	Funcionamiento	Valor devuelto
handle_register	Lee el nombre de usuario. Bloquea el mutex y recorre la lista global para comprobar si ya existe. Si no existe, crea un nuevo nodo User con status=0, last_msg_id=0, pending_msgs=NULL y lo inserta al inicio de la lista.	0 si éxito, 1 si el nombre del usuario ya está registrado o 2 si hay un error.
handle_unregister	Lee el nombre de usuario. Busca y elimina el nodo de la lista enlazada (actualizando prev->next). Antes de liberar la memoria del usuario, recorre su lista de mensajes pendientes y los libera también.	0 si éxito, 1 si el usuario no existe.
handle_connect	Lee el nombre de usuario y el puerto de escucha (como cadena, se convierte con atoi). Busca el usuario y si lo encuentra actualiza status a 1, copia la IP y le asigna el puerto. Luego recorre la lista de mensajes pendientes y envía	0 si éxito, 1 si el usuario no está registrado, 2 si el usuario ya está conectado, o 3 si error.

	cada mensaje al cliente usando send_message_to_client() o send_attach_to_client(), dependiendo de si hay filename o no (aunque esta segunda función la implemnetamos en la segunda parte). Si el envío es exitoso, se notifica al remitente mediante un SEND_MESS_ACK.	
handle_disconnect	Lee el nombre de usuario. Busca el usuario y si está conectado pone status a 0, limpia la IP y puerto. No borra los mensajes pendientes.	0 si éxito, 1 si el usuario no está registrado. 2 si el usuario no estaba conectado.
handle_users	Lee el nombre del usuario que realiza la petición. Comprueba que el usuario esté conectado. Luego cuenta los usuarios conectados (con status igual a 1) y envía al cliente un byte 0, una cadena con el número de conectados, y por cada usuario conectado envía una cadena con la información de cada usuario.	0+número de usuarios+cadena con info de cada usuario(implementado en la parte2) si éxito, 1 si el usuario que solicita la acción no está conectado, o 2 si error.
handle_send	Lee el nombre del remitente, el destinatario y el mensaje. Busca el destinatario y el remitente, el cual debe estar conectado, y si los encuentra, incrementa la variable last_msg_id del usuario, crea un mensaje y lo añade a la lista de mensajes pendientes del destinatario. Si el destinatario está conectado intenta enviarle el mensaje y si es exitoso se elimina de la lista de pendientes y notifica al remitente con SEND_MESS_ACK.	0 si éxito, junto con el ID, 1 si el usuario destinatario o remitente no existe o está desconectado, y 2 si hay un error.
handle_quit	Lee el nombre del usuario y lo busca. Si existe, asigna el valor 0 a la variable status (lo marca como desconectado), limpia la IP y el puerto. Esta función no borra su cuenta ni sus mensajes pendientes. Esta función hace que el cliente cierre la conexión.	No envía código de retorno, ya que el cliente cierra la conexión tras enviar QUIT.

Además para el envío de un mensaje a un cliente utilizamos una función auxiliar "send_messge_to_client()" que se conecta a la IP y puerto del cliente destino, envía la

cadena “SEND_MESSAGE\0” , luego el remitente, el ID y el mensaje. A continuación cierra la conexión y retorna 0 si no hubo ningún error, o -1 en caso contrario.
También tenemos la función auxiliar “read_str()” que lee del socket carácter a carácter hasta encontrar un ‘\0’ o alcanzar la longitud máxima de la cadena (maxlen-1). Almacena los bytes en un buffer y añade el terminador. Devuelve el número de bytes leídos si no ocurre ningún error o -1 en caso contrario, finalizando la conexión.

- **Cliente**

El cliente de nuestro programa es multihilo y sigue el protocolo que explicamos después. El hilo principal se encarga de leer los comandos del usuario y enviar peticiones al servidor, mientras que los hilos secundarios, que son hilos de escucha, reciben los mensajes que entran de otros usuarios a través del servidor.

Todas las funciones de operación del cliente siguen una misma estructura. Esta es, crear un nuevo socket TCP, conecta con el servidor, envía la cadena de operación y los parámetros requeridos. Luego recibe un byte de respuesta, que es el código de estado, y según su valor, imprime el mensaje que corresponda. Si la operación es SEND o SENDATTACH y el código es 0, lee además una cadena con el identificador del mensaje. Por último, en todas las funciones cerramos el socket.

Las partes distintas o adicionales de las funciones a esta estructura las siguientes:
-En la función connect, antes de enviar la solicitud, creamos un socket de escucha en un puerto libre. Luego inicializamos el hilo listen_thread y envía al servidor el comando CONNECT y el puerto de escucha. Si la conexión falla, se detiene al hilo de escucha y se cierra el socket.
-En la función disconnect y quit se detiene el hilo de escucha.
-En la función users, se actualiza el diccionario interno de usuarios conectados con la IP y puerto de cada usuario conectado, necesario para la transferencia P2P de la parte 2.

- **Protocolo**

El protocolo diseñado para esta aplicación se sitúa en el nivel de aplicación sobre el protocolo de transporte TCP. Define una arquitectura donde el servidor actúa como intermediario y gestor de estados, facilitando la comunicación entre clientes. El intercambio de información se basa en una combinación de cadenas de caracteres (strings) para los datos y códigos binarios para los estados de control.

<u>Flujo de interacción:</u>

1. **Solicitud del Cliente:** Cuando el usuario introduce un comando en la terminal, el cliente (client.py) serializa la operación. Primero envía el nombre del comando (ej. REGISTER, CONNECT, SEND) seguido de sus argumentos. Para que el servidor pueda identificar el final de cada campo mediante la función read_str(), cada cadena de texto se envía terminada en el carácter nulo ('\0').

2. **Procesamiento en el Servidor:** El servidor (servidor.c) recibe la cadena identificadora de la operación y, mediante un flujo de control, invoca la función lógica correspondiente implementada en claves.c.
3. **Respuesta de Estado:** Una vez ejecutada la lógica, la función devuelve un código de estado de 1 byte (un uint8_t). Este código se envía de vuelta al cliente para indicar el resultado de la operación: Si devuelve 0, la petición ha sido un éxito, si recibe 1 o 2, se ha cometido errores específicos según qué función(es) se hayan ejecutado
4. **Confirmación al Usuario:** El cliente (client.py) recibe este byte, lo interpreta y despliega un mensaje descriptivo por la terminal, informando al usuario del éxito o fracaso de su petición.

2.1. Parte 2

Para permitir la transferencia de archivos mediante P2P, la normalización de mensajes y la monitorización de mensajes por remoto, se ha extendido la funcionalidad del sistema añadiendo funciones y programas nuevos al código existente de la parte 1.

- **Transferencia de ficheros**

Ya se tenía una estructura previa creada para gestión de mensajes entre clientes que permitía a un cliente mandar a otro mensajes, ahora se pide expandir esa funcionalidad para permitir el envío de ficheros. Sin embargo y a diferencia de lo ya implementado, se pide que se realice mediante una comunicación P2P, en la cual el servidor únicamente facilita la comunicación entre los clientes y únicamente pasa por él los nombres de los ficheros y los identificadores de usuario.
La función getfile() no se implementa en el servidor, se implementa directamente en el cliente, ya que no debe intervenir el servidor para el envío de ficheros entre clientes según el enunciado.

Nombre función	Funcionamiento	Valor devuelto
handle_sendattach	Lee el nombre del remitente, el destinatario, el mensaje y el nombre del fichero. Busca el destinatario y el remitente, el cual debe estar conectado, y si los encuentra, incrementa el campo last_msg_id del remitente, crea un mensaje con un nombre de fichero adjunto y lo añade a la lista de mensajes pendientes del destinatario. Si el usuario no está conectado se almacena el mensaje y se envía después, igual que con el send de	0 si éxito, junto con el ID, 1 si el usuario destinatario o remitente no existe o está desconectado, y 2 si hay errores de memoria u otra cosa.

	la parte 1. Una vez que el destinatario recibe el mensaje adjunto, el servidor notifica al remitente con la cadena SEND_MESS_ATTACH_ACK, el ID y el nombre del fichero.	
getfile	Lee el nombre del usuario que tiene el fichero, el nombre del fichero remoto y el nombre del fichero local. Busca al usuario que tiene el fichero, el cual debe estar conectado, y si no lo encuentra en la lista local de usuarios conectados, pide al servidor que lo busque con users(). Si lo encuentra, obtiene su IP y su puerto y se conecta mediante un socket al puerto del cliente. Se envía el código getfile con el nombre del archivo remoto. El receptor intercepta con el hilo de escucha el código de operación y el nombre del archivo, lo busca y lo reenvía por el mismo puerto. El emisor recibe una respuesta, si es correcta recibe el contenido del fichero poco a poco, que se va almacenando en el archivo local provisto al principio.	Si tiene éxito (0), no devuelve un valor específico por pantalla más allá de terminar la descarga y guardar el archivo en la ruta local indicada (local_file). Si el usuario no está conectado,y tras buscarlo con users() sigue sin estarlo, o directamente no existe, devuelve "FILE TRANSFER FAILED, user not connected." Si hay un error de red o fichero, se captura la excepcion y se imprime por pantalla el error correspondiente.

En esta segunda parte implementamos la función "send_attach_to_client()" para el envío de un mensaje con fichero adjunto a un cliente, que se conecta a la IP y puerto del cliente destino, envía la cadena "SEND_MESSAGE_ATTACH\0" , luego el remitente, el ID, el mensaje y el fichero adjunto. A continuación cierra la conexión y retorna 0 si no hubo ningún error, o -1 en caso contrario.

Además, la operación USERS se ha modificado para que el servidor devuelva, por cada usuario conectado, una cadena con formato "nombre :: IP :: puerto". El cliente almacena esta información en el diccionario de los usuarios conectados.

- **Servicio web**

El objetivo de este componente es la normalización de los mensajes, delegando el procesamiento de texto en un servicio externo especializado. Este enfoque permite desacoplar la lógica de edición de los nodos principales (cliente y servidor central), siguiendo un modelo de arquitectura orientada a servicios. La limpieza de los mensajes

se realiza mediante un servicio web SOAP implementado en Python con las librerías Spyne (servidor) y Zeep (cliente).

- **Protocolo SOAP y Estructura**

El protocolo SOAP (Simple Object Access Protocol) utiliza XML para estructurar el intercambio de información. Se caracteriza por ser un protocolo estricto y tipado, lo que garantiza la integridad de los datos en entornos distribuidos heterogéneos. El servidor se implementa como un proceso independiente que gestiona las peticiones mediante una estructura de tres niveles:

1. **Definición de la Interfaz (Spyne):** Utilizamos la librería Spyne para definir el servicio mediante una clase que hereda de ServiceBase. El uso del decorador @rpc es fundamental, ya que define el contrato de la operación: especifica que el método recibe un tipo de dato Unicode (parámetro de entrada) y devuelve igualmente un **Unicode** (valor de retorno).
2. **Lógica de Negocio:** Corresponde a la función interna que ejecuta la tarea de "limpieza". Esta lógica procesa la cadena recibida para eliminar espacios en blanco redundantes y devuelve el mensaje normalizado.
3. **Exposición y Contrato WSDL:** El servidor genera dinámicamente el WSDL (Web Services Description Language), un documento XML que actúa como un contrato de interfaz. Este se genera automáticamente cuando Spyne inspecciona los métodos decorados con @rpc y proyecta un esquema XML que define los tipos de datos, así como los mensajes de solicitud (Request) y respuesta (Response).

- **Infraestructura de Transporte y Despliegue**

Para que el servicio sea accesible a través de la red, se requiere una infraestructura de transporte, en este caso HTTP.

-Wrapper de Aplicación (WSGI): Dado que Spyne gestiona la lógica del protocolo pero no es un servidor web por sí mismo, empleamos WsgiApplication. Este *wrapper* actúa como adaptador para que la lógica SOAP sea compatible con el servidor web.

-Servidor de Transporte: Utilizamos la librería wsgiref para levantar un servidor en el puerto 8000. Cuando este servidor recibe una petición HTTP, el *wrapper* extrae el mensaje (el sobre SOAP) y se lo entrega a Spyne para su procesamiento. Finalmente, el servidor publica el contrato en la URL http://localhost:8000/?wsdl, permitiendo que el cliente se autoconfigure.

Como spyne es una librería lógica de protocolos, no un servidor web en sí mismo, se necesita un wrapper de aplicación para compatibilizar la librería Spyne con el servidor web. Empleamos WsgiApplication para que una vez el servidor web obtenga una

petición, el wrapper extraiga el contenido (el sobre SOAP) y así Spyne pueda generar y devolver un archivo de definición XML.

- **Servicio RPC: Registro de actividad (Logs)**

Esta extensión añade un servicio de monitorización basado en el modelo ONC-RPC. Su función es registrar en una entidad externa cada operación que los usuarios realizan en el sistema, permitiendo un seguimiento centralizado de la actividad.

Para garantizar la interoperabilidad y la correcta serialización de los datos, se define la interfaz mediante un archivo IDL.

Estructura de datos: Se ha definido la estructura log_request que empaqueta tres cadenas de caracteres (strings): el nombre del usuario, el código de la operación y el nombre del fichero (parámetro que se envía vacío si la operación no es SENDATTACH).

Contrato del servicio: Se define el programa LOG_PROG con la versión LOG_VERS y un único procedimiento remoto PRINT_LOG. Este procedimiento es el que el servidor RPC ejecutará cada vez que sea invocado.

En el archivo claves.c, se ha implementado la función auxiliar call_rpc_log. Esta función realiza las siguientes tareas:

1. **Localización del servicio:** Obtiene la dirección IP del servidor RPC mediante la variable de entorno LOG_RPC_IP.
2. **Instanciación del cliente:** Utiliza la función clnt_create para establecer un canal de comunicación sobre el protocolo **TCP** con el servidor remoto.
3. **Invocación remota:** Rellena la estructura log_request con los datos de la operación actual y llama a la función print_log_1. Esta llamada es transparente para la lógica del servidor, comportándose como una función local aunque se ejecute en otro proceso o máquina.

El servidor RPC (log_rpc_server.c) permanece a la escucha de peticiones. Al recibir una invocación de PRINT_LOG, extrae los datos de la estructura e imprime por salida estándar en nombre del usuario seguido de la operación y, en caso de darse, el nombre del fichero.

3. Descripción compilación y obtener ejecutables

Lo primero que hacemos para ejecutar nuestro programa es abrir Docker y abrir el trabajo en el contenedor. Cuando ya tenemos toda la parte de Docker lista, escribimos en la terminal el comando "`make`", que ejecuta el Makefile que genera los ficheros RPC a partir del archivo log_rpc.x, compila el servidor y cliente, junto con los archivos generados con el RPC, y se crea el ejecutable del servidor. Además se compila el servidor RPC, junto con sus archivos creados y se crea el ejecutable log_rpc_server. Por

tanto ejecutar make nos crea el ejecutable servidor, que es el servidor principal de mensajeria, y el ejecutable log_rpc_server que es el servidor RPC que registra las operaciones.
Para ejecutar el servicio web escribimos en la terminal el comando "`python3 ws_server.py`", cuyo servicio se ejecuta en el localhost 8000 y permanece a la espera de peticiones SOAP.
Para ejecutar el servidor RPC escribimos en la terminal el comando "`./log_rpc_server`", que queda en ejecución escuchando las peticiones RPC y no produce salida hasta que recibe operaciones.
El servidor de mensajeria lo ejecutamos definiendo primero en la terminal la variable de entorno "`export LOG_RPC_IP=127.0.0.1`" que es la IP del servidor RPC. Luego lanzamos el servidor con "`./servidor 8888`" donde utilizamos por ejemplo el número de puerto 8888.
Por último para ejecutar un cliente, hay que escribir en la terminal el comando "`python3 client.py -s 127.0.0.1 -p 8888`", en el que indicamos la IP y puerto del servidor de mensajería.

4. Batería de pruebas

Para probar el correcto funcionamiento de nuestro programa, hemos realizado distintos tipos de pruebas, en las que tenemos 5 terminales. Estas son la terminal de log_rpc_server, la terminal de ws_server.py, la terminal del servidor, la terminal del usuario Ana y la terminal del usuario Luis. Todas las pruebas las hemos realizado con estas 5 mismas terminales.

<u>-Prueba de registro y conexión de dos usuarios:</u>
En esta prueba registramos y conectamos a dos usuarios. Además tratamos de registrar 2 veces al mismo usuario para que nos de el error de nombre de usuario en uso.

Tras iniciar los servidores, desde la terminal de Ana enviamos un REGISTER ana y se registra correctamente. Luego lo volvemos a escribir y vemos que aparece el mensaje de "USERNAME IN USE" que es el esperado. Luego hacemos un CONNECT ana con el que aparece el mensaje de que ha salido bien, y luego escribimos USERS, viendo que aparece solo ana, lo cual es correcto. Además el Servidor RPC verificó la recepción de las llamadas remotas. En la terminal del servidor de logs aparecieron de forma secuencial las líneas ana REGISTER, ana REGISTER y ana CONNECT. Luego apareció "REGISTER ana OK" y "CONNECT ana OK", y ante el segundo intento de registro, devolvió el código de error correspondiente, confirmando que la búsqueda en la lista enlazada de usuarios protegida por el mutex funciona correctamente. Por último hacemos lo mismo con luis (le registramos y conectamos) y al hacer USERS de nuevo vemos que aparecen ana y luis.

-Prueba de envío de mensaje simple (entrega inmediata):
En esta prueba Ana le envía un mensaje simple a Luis, es decir, sin fichero adjunto.

Tras mandar desde la terminal de ana el mensaje “SEND luis Hola Luis” vemos que en la terminal de Ana aparece el mensaje de “SEND OK - MESSAGE 1” y tiempo después el de confirmación (SEND MESSAGE 1 OK), lo que confirma que el servidor ha procesado y almacenado temporalmente el mensaje. En la terminal de Luis aparece el mensaje de que le ha llegado, ya que está conectado, lo que demuestra que el hilo de ejecución en el servidor encargado de la petición de ana abrió con éxito un nuevo socket TCP hacia el puerto de escucha de Luis para entregarle el contenido. Luego vemos en la terminal del servidor que se imprime el mensaje “SEND MESSAGE 1 FROM ana TO luis” y en el servidor RPC se registra la línea “ana SEND” lo que es correcto.

-Prueba de envío de mensaje con fichero adjunto (entrega inmediata):
En esta prueba Ana envía a Luis un mensaje con un fichero adjunto. Ambos están conectados.

Tanto ana como luis están registrados y conectados, tenemos un archivo que se llama fichero.txt que contiene el texto “esto es un mensaje de prueba” y mandamos desde la terminal de ana el mensaje “SENDATTACH luis fichero.txt Este es un mensaje con fichero adjunto”. Podemos ver que en la terminal de Ana aparece el mensaje de que se envía el mensaje (SENDATTACH OK - MESSAGE 2) y tiempo después el de confirmación (SENDATTACH MESSAGE 2 fichero.txt OK), lo que es correcto. Además en la terminal de Luis le vemos que le llega el mensaje con el formato pedido, que es “MESSAGE 2 FROM ana Este es un mensaje con fichero adjunto END FILE fichero.txt”. También vemos que en la terminal del servidor aparece el mensaje “SEND MESSAGE FROM ana TO luis ATTACHED fichero.txt” y en la terminal del servidor RPC aparece “ana SENDATTACH fichero.txt”, lo que es correcto.

-Prueba de transferencia de fichero (GETFILE) entre usuarios:
En esta prueba Luis coge el archivo que le ha mandado antes Ana y ambos están conectados.

Tanto ana como luis están registrados y conectados y ana ya le ha mandado un mensaje con archivo adjunto correspondiente a la prueba anterior. En la terminal de luis ponemos el mensaje “GETFILE ana fichero.txt mi_copia_fichero.txt” y vemos que aparece el mensaje de que ha salido bien la operación en la terminal de Luis (GETFILE OK) y se crea el archivo mi_copia_fichero.txt con el mismo contenido que el archivo fichero.txt. En esta prueba también probamos a ejecutar el mismo comando pero con un archivo que no existe y nos da el error esperado. Además en la terminal del servidor y la de RPC no aparece ningún mensaje nuevo ya que GETFILE es una transferencia directa entre clientes por lo que no participan en la transferencia del contenido del fichero.

-Prueba de mensaje a usuario desconectado (entrega diferida):
En esta prueba desconectamos a Luis y enviamos mensajes desde la terminal de Ana.

Lo primero que hacemos en esta prueba es desconectar a luis con “DISCONNECT luis” y mandamos desde la terminal de ana dos mensajes: “SEND luis Esto es un mensaje diferido, no debe aparecer” y “SENDATTACH luis fichero.txt ESto es un fichero diferido”. Vemos que no aparece nada en la terminal de Luis y a Ana no le llegan los mensajes de confirmación, lo que es correcto. Tras conectar a Luis con “CONNECT luis”, vemos que recibe los mensajes que le había mandado Ana y ella recibe los mensajes de confirmación (SEND MESSAGE 3 OK y SENDATTACH MESSAGE 4 fichero.txt OK) , por lo que demuestra que el programa funciona correctamente. Además aparecen en las terminales del servidor y RPC los mensajes correspondientes.

-Prueba de servicio web (normalización de espacios):
En esta prueba Ana mandará un mensaje a Luis con espacios entre medias. Tanto ana como luis están registrados y conectados.

Para esta prueba tenemos a ambos usuarios conectados y enviamos desde la terminal de ana el mensaje “SEND luis Hola pero con muchos espacios”. Tras enviarlo, vemos que a Luis le llega el mensaje sin los espacios (“MESSAGE 5 FROM ana Hola pero con muchos espacios END”), y en la terminal del Servicio Web, aparece la transformación que ha realizado, que es correcta, lo que demuestra que la comunicación entre el programa en C y el ws_server.py es es correcta. Además aparecen en las terminales del servidor y RPC los mensajes correspondientes.

-Prueba de casos extremos y gestión de errores:
En esta prueba comprobamos que aparezca el mensaje correspondiente cuando el usuario no existe y se intenta mandar un mensaje, cuando conectamos un usuario que ya está conectado, cuando quieres desconectar un usuario que ya estaba desconectado y que si haces QUIT el cliente termina.

Para esta prueba, desde la terminal de ana tratamos de mandar un mensaje a un usuario que no existe con “SEND inexistente hola” y nos aparece el mensaje de error “SEND FAIL, USER DOES NOT EXIST”. Tambien hacemos lo mismo con SENDATTACH y aparece el mensaje esperado. Luego tratamos de conectar un usuario previamente ya conectado con “CONNECT luis” apareciendo el mensaje de “USER ALREADY CONNECTED”. Luego desconectamos a Luis con “DISCONNECT luis” que aparece el mensaje de “DISCONNECT OK” y volvemos a mandar el mensaje de desconectar a luis, apareciendo el mensaje de “DISCONNECT FAIL, USER NOT CONNECTED”. Por último, utilizamos QUIT tanto en la terminal de ana como en la de luis, y vemos que de desconectan ambos usuarios correctamente. Además aparecen en las terminales del servidor y RPC los mensajes correspondientes.

-Conclusiones de las pruebas:
Tras comprobar que los identificadores de mensaje funcionan correctamente (cada usuario mantiene su propia secuencia de IDs, iniciando en 1 e incrementándose bien), que los mensajes simples y con fichero adjunto se entregan inmediatamente cuando el destinatario está conectado y se almacenan cuando está ausente, llegando los mensajes de confirmación correspondientes, se recupera el fichero adjunto correctamente, se normalizan los mensajes, aparecen bien los logs de RPC, aparecen los errores esperados y se pueden atender múltiples clientes, demostramos que el programa funciona correctamente y cumple con lo pedido en la práctica.

5. Conclusiones

El desarrollo de esta práctica nos ha permitido diseñar e implementar un sistema distribuido de mensajería con las siguientes funcionalidades: envío de mensajes de texto, transferencia de ficheros entre usuarios mediante comunicación P2P, normalización de mensajes a través de un servicio web SOAP y registro centralizado de operaciones mediante RPC.
Esta práctica ha sido muy completa y gracias a ella hemos aprendido a utilizar distintas tecnologías, diferentes tipos de comunicación y hemos practicado los diferentes tipos de lenguajes de programación.
En cuanto a los problemas encontrados, en un principio se realizó un socket no por operación, si no por cliente, de forma que e tenía un socket abierto por el que se mandaban las peticiones del cliente, con 8 hilos procesando clientes en paralelo, 7 de ellos estarían bloqueados cada vez que uno hace log. El paralelismo que te da el pool de hilos quedaría anulado justo en el punto de logging. De tal forma que no se permitiría tener a más de 8 clientes activos, bloqueando todas las otras peticiones.
Cuando concluimos la primera parte del trabajo nos dimos cuenta del error y cambiamos para permitir un mayor flujo de peticiones y más conexiones de clientes.